Curia-Lite: Minimizing Model Size for Keyword and Temporal Query Extraction

Will Adair
University of Nebraska – Lincoln
Email: wadair2@huskers.unl.edu

Will Glover
University of Nebraska – Lincoln
Email: wglover2@huskers.unl.edu

Rishi Krishna
University of Nebraska – Lincoln
Email: rkrishna2@huskers.unl.edu

Michael Dejournett
University of Nebraska – Lincoln
Email: mdejournett2@huskers.unl.edu

Amy Nguyen
University of Nebraska – Lincoln
Email: anguyen105@huskers.unl.edu

Maya Wilson
University of Nebraska – Lincoln
Email: mwilson75@huskers.unl.edu

Krishnaraj Ganesan
University of Nebraska – Lincoln
Email: krishnarajganesan@nebraska.edu

***Abstract*—This paper frames Curia as an efficiency study centered on model-size minimization for event query understanding. We investigate how far the model can be reduced while preserving two core capabilities: keyword extraction and temporal grounding. Using a two-path inference pipeline—a large remote baseline (`gemma-3-27b-it`, 27B parameters via the Gemini API) and a local in-context learning path (`Qwen2.5-0.5B-Instruct`)—we evaluate six model candidates ranging from 248M to 27B parameters on a rigorous benchmark of 72 temporal queries derived from real UNL event data. The 27B Gemini path achieves 100% parse success, 100% temporal exact-match accuracy, and a Mean Reciprocal Rank (MRR) of 0.650. The local 500M Qwen2.5-0.5B path achieves 23.6% parse success and 19.4% date exact-match at 6.9 s CPU latency, while all other sub-1B candidates fail to produce structured output. Results reveal asymmetric degradation: keyword retention holds longer than temporal extraction as capacity decreases, and a structured output ceiling appears below 1B parameters on consumer CPU. A lightweight hybrid retrieval module (`all-MiniLM-L6-v2`, 22M parameters) augments keyword output for both paths at under 50 ms, offering a practical way to extend semantic coverage without increasing the primary model size.**

***Index Terms*—Lightweight Models, Temporal Grounding, Information Retrieval, Query Parsing, Edge Inference**

I. Introduction

Event search systems depend on two query-understanding capabilities: extracting intent-bearing keywords and grounding temporal expressions into structured date and time constraints. A user typing "free food next Thursday after 3pm" has simultaneously specified a topic, a relative date anchor, and a time lower bound. Converting this free-form input into structured filters drives retrieval quality.

In practice, these systems must often operate under tight memory and latency budgets. This raises a central design tension: how far can the underlying model be reduced before extraction capability degrades enough to harm retrieval? Prior work on neural scaling laws [1], [2] suggests that capabilities do not decline uniformly across tasks as capacity decreases, but the exact pattern depends on task difficulty and output structure requirements.

This paper studies that tension directly in Curia, a campus event search system for UNL. Rather than targeting architectural novelty, we evaluate whether model-size reduction can preserve the behaviors needed for downstream retrieval. We focus on two research questions:

1) Can model size be substantially reduced while retaining keyword extraction performance?
2) Can model size be reduced while retaining temporal extraction performance?

Our experiments show asymmetric degradation. Keyword extraction remains partially functional at 500M parameters, while temporal structured output collapses entirely below that scale. A sub-1B structured-output ceiling appears where models either time out or produce malformed JSON on our 72-query rigorous benchmark.

This paper makes three contributions:

- **Task-specific size analysis**: An empirical comparison of six model candidates (248M to 27B) on keyword extraction and temporal grounding within the same event-search pipeline, evaluated on a benchmark of 72 labeled temporal queries.
- **Failure characterization**: Documentation of how and where temporal extraction breaks down as model capacity decreases, including the role and limits of rule-based fallbacks and the reasons two earlier candidates were abandoned.
- **Deployment guidance**: Practical guidance for selecting model scales based on feature priorities, including the case for a small hybrid retrieval module as a lightweight substitute for LLM-based keyword expansion.

II. Related Work

A. Temporal Information Extraction

Traditional temporal extraction systems use hand-crafted rule sets and lexicons, exemplified by the TIMEX3 annota-

tion scheme [3] and systems such as HeidelTime [4]. These are interpretable but fragile on linguistic variation. Neural approaches fine-tuned on TimeBank [5] improve recall on news documents but inherit a document-centric assumption: temporal expressions are anchored to an article's publication date, not to the user's present moment. As we show in Section IV, a model trained for document-side TIMEX3 tagging fails completely when applied to query-side resolution against a live reference date.

B. Neural Scaling Laws

Scaling law research [1], [2] shows that model performance on many tasks grows predictably with parameter count, but emergent capabilities can appear or disappear sharply at specific thresholds. Structured output generation—producing well-formed JSON with correct field values—is particularly sensitive to capacity, since it requires simultaneous control of format, arithmetic reasoning, and factual grounding.

C. Few-Shot and In-Context Learning

In-context learning (ICL) [6] allows language models to perform new tasks from a handful of demonstrations in the prompt, without any gradient updates. Chat-template prompting—where examples are formatted as `user`/`assistant` turns [7]—improves instruction-following in decoder-only models. We use this approach with `Qwen2.5-0.5B-Instruct`, providing 4 exemplars for temporal extraction and 3 for keyword expansion, with no weight updates.

D. Hybrid Dense Retrieval for Keyword Expansion

Dense retrieval models [8] and sentence embedding models [9] enable semantic matching beyond keyword overlap. For keyword expansion specifically, nearest-neighbor lookup over a domain vocabulary is a lightweight technique to supplement LLM output without a full retrieval-augmented generation pipeline. Our MiniLM hybrid module follows this pattern and adds negligible latency.

E. Hybrid Rule and Learning-Based Systems

Combining neural models with rule-based fallbacks improves robustness on out-of-distribution inputs. For temporal grounding in particular, deterministic libraries such as `dateparser` [10] provide reliable coverage of explicit date strings and common relative expressions that small neural models handle poorly. Applying the same fallback policy across all model sizes allows us to compare degradation under a stable hybrid configuration.

III. Methodology

A. Divergence-Centered Study Design

Our design measures how capability diverges across the two extraction tasks as model size decreases. We evaluate with two coupled objectives: preserve keyword extraction as far as possible, and measure precisely where temporal extraction degrades. No quantization, pruning, or distillation is applied; model family scale is the sole experimental variable. This isolates capacity as the primary factor and avoids confounding from additional compression techniques.

B. Task Outputs

Given a natural language query q, Curia produces two outputs:

- **Keywords**: An expanded set of intent-bearing terms for retrieval matching.
- **Temporal bounds**: A JSON object with four fields — `date_from`, `date_to`, `time_from`, `time_to` — or `null` when no constraint is present.

For example, "morning concerts next weekend" should yield:

```
{
  "date_from": "2026-05-02",
  "date_to":   "2026-05-03",
  "time_from": "06:00",
  "time_to":   "12:00",
  "keywords":  ["concert", "music",
                "performance", "band", "show"]
}
```

C. Divergence Metric

Let $s_{\max}$ denote the reference model (27B Gemini) and s any smaller model. We normalize each task score to its value at $s_{\max}$:

$$\hat{F1}_{\text{kw}}(s) = \frac{F1_{\text{kw}}(s)}{F1_{\text{kw}}(s_{\max})}, \quad \hat{EM}_{\text{temp}}(s) = \frac{EM_{\text{temp}}(s)}{EM_{\text{temp}}(s_{\max})}.$$

Performance divergence is then:

$$D_{\text{perf}}(s) = \hat{F1}_{\text{kw}}(s) - \hat{EM}_{\text{temp}}(s).$$

Positive $D_{\text{perf}}(s)$ means keyword extraction is retained better than temporal extraction at size s.

D. Inference Paths

a) Gemini Remote Path (27B baseline): A single structured prompt is sent to `gemma-3-27b-it` via the `google-genai` SDK. The prompt includes the current date and asks the model to return a JSON object with temporal fields and an expanded keyword list. This path serves as the $s_{\max}$ reference for divergence normalization. Mean benchmark latency: 7.9 s (p95: 8.7 s).

b) Local Two-Stage Path: `Qwen2.5-0.5B-Instruct` (500M parameters, ≈954 MB) runs on CPU via the HuggingFace `transformers` library using two sequential prompts:

- *Stage 1 — Temporal extraction*: 4 few-shot exemplars with chain-of-thought reasoning prefixes; current date injected into the system message.
- *Stage 2 — Keyword expansion*: 3 few-shot exemplars mapping queries to 20–25-term keyword lists.

Both stages use greedy decoding (`do_sample=False`), max 512 new tokens.

c) Controlled Fallback Layer: The same fallback policy applies at every model scale: (1) `dateparser` for date fields when LLM JSON is malformed or absent; (2) a time-of-day keyword lookup for time fields (morning → 06:00–12:00, afternoon → 12:00–17:00, evening → 17:00–21:00); (3) null bounds if both fail. Applying this uniformly allows divergence to be attributed to model capacity rather than differing recovery policies.

E. MiniLM Hybrid Retrieval

`all-MiniLM-L6-v2` (22M parameters) embeds the query and a pre-computed vocabulary of 384 domain terms drawn from the backend keyword corpus. The top-k nearest neighbors by cosine similarity are appended to the LLM's keyword list. The corpus is embedded once at startup; retrieval adds under 50 ms per query. This module is active for all model paths.

F. Event Scoring

The backend ranks events by weighted field score:

$$\text{score}(e, Q) = \sum_{t \in Q} w_f \cdot \mathbf{1}[t \in \text{field}_f(e)] \cdot \text{pos}(t, Q)$$

where $w_f \in \{4, 2, 1\}$ for name, venue/category, and description/tags respectively, and $\text{pos}(t, Q)$ is a position bonus giving higher weight to terms that appear earlier in the query. Temporal bounds are applied as hard pre-filters before scoring.

G. Models Evaluated

Table I lists all six candidates evaluated in this work. `all-MiniLM-L6-v2` is used exclusively for hybrid retrieval augmentation and is not evaluated as a standalone grounder.

TABLE I
MODEL CANDIDATES

Model	Backend	Params
`gemma-3-27b-it`	Gemini API	27B
`Qwen2.5-0.5B-Instruct`	HF local	500M
`Qwen2.5-1.5B-Instruct`	HF local	1.5B
`TinyLlama-1.1B-Chat-v1.0`	HF local	1.1B
`google/flan-t5-base`	HF local	250M
`LaMini-Flan-T5-248M`	HF local	248M
`all-MiniLM-L6-v2`	HF local (retrieval)	22M

IV. EXPERIMENTS

A. Benchmark Dataset

We construct an evaluation set of 72 temporal queries derived from real UNL events (scraped via the Curia pipeline). Each case includes: a natural language query; ground-truth `date_from`, `date_to`, `time_from`, `time_to`; a set of expected keywords; and one or more relevant event URLs. Cases are grouped into A/B/C variant clusters that test different phrasings of the same underlying temporal constraint. The full dataset is stored at `testing/benchmark/datasets/queries_rigorous_temporal.json`.

B. Metrics

- **Parse Success Rate**: Fraction of queries with valid, parseable JSON output
- **Date Exact Rate**: Exact match on both `date_from` and `date_to`
- **Time Exact Rate**: Exact match on both time fields (over cases with ground-truth time bounds)
- **Keyword F1**: Token-level F1 between predicted and expected keyword sets
- **Top-k Hit Rate**: Ground-truth event URL in top-k retrieved results
- **MRR**: Mean Reciprocal Rank of the ground-truth event across all results
- **Latency**: Mean and p95 wall-clock time in milliseconds (CPU, 4-core)

C. Main Results

Table II reports accuracy across all six models on 72 temporal queries.

The Gemini path dominates all accuracy metrics. Among the five local candidates, only `Qwen2.5-0.5B-Instruct` achieves non-trivial parse success (23.6%) and date extraction (19.4%), but produces zero correct time-of-day outputs. `flan-t5-base` achieves 100% parse success (seq2seq output is always syntactically well-formed) but zero accuracy on every substantive metric, confirming the known failure mode of zero-shot structured JSON generation at 250M parameters.

D. Divergence Analysis

Using $D_{\text{perf}}(s) = \hat{F1}_{\text{kw}}(s) - \hat{EM}_{\text{temp}}(s)$ with the 27B Gemini model as $s_{\max}$, we observe positive divergence at all sub-27B scales: keyword F1 retains 71% of baseline at 500M parameters ($\hat{F1}_{\text{kw}} = 0.71$) while temporal exact-match retains 0% at the same scale ($\hat{EM}_{\text{temp}} = 0.0$), giving $D_{\text{perf}} = 0.71$. This confirms the asymmetric degradation hypothesis: temporal extraction is more sensitive to capacity reduction than keyword extraction under the same in-context learning protocol.

E. Computational Efficiency

`flan-t5-base` is fastest (402 ms) but produces no useful output. Among models with non-trivial accuracy, Gemini (7.9 s API) and `Qwen2.5-0.5B` (6.9 s CPU) are comparable in latency despite the 54× parameter ratio, reflecting network round-trip overhead versus CPU token generation. The 1.5B Qwen model is 3× slower than Gemini (23.9 s) with zero accuracy, placing it off the quality-latency frontier entirely. Figure 4 visualizes the tradeoff.

F. Benchmark Visualizations

G. Scale Ablation

In a separate 8-query exploratory evaluation (Table IV), we compared `Qwen2.5-0.5B` and `Qwen2.5-1.5B` directly. The 1.5B model achieved 3/7 date matches and 2/2 time-of-day matches versus 0.5B's 1/7 and 0/2, but at ≈129 s per query CPU latency — 14× slower than Gemini and impractical for

TABLE II
QUERY GROUNDING ACCURACY ON 72 RIGOROUS TEMPORAL QUERIES

Model	Parse Success	Date Exact	Time Exact	KW F1	Top-3 Hit	MRR
`gemma-3-27b-it` (Gemini)	**1.000**	**1.000**	**1.000**	**0.066**	**0.764**	**0.650**
`Qwen2.5-0.5B-Instruct`	0.236	0.194	0.000	0.047	0.069	0.071
`Qwen2.5-1.5B-Instruct`	0.000	0.000	0.000	0.000	0.000	0.000
`TinyLlama-1.1B-Chat-v1.0`	0.000	0.000	0.000	0.000	0.000	0.000
`flan-t5-base`	1.000	0.000	0.000	0.000	0.000	0.000
`LaMini-Flan-T5-248M`	0.000	0.000	0.000	0.000	0.000	0.000

TABLE III
CPU INFERENCE LATENCY (72 QUERIES, 4-CORE)

Model	Params	Mean (ms)	p95 (ms)
`flan-t5-base`	250M	**402**	**396**
`LaMini-Flan-T5-248M`	248M	1,213	919
`Qwen2.5-0.5B-Instruct`	500M	6,911	7,995
`TinyLlama-1.1B-Chat-v1.0`	1.1B	12,643	12,700
`gemma-3-27b-it` (API)	27B	7,917	8,741
`Qwen2.5-1.5B-Instruct`	1.5B	23,898	24,139

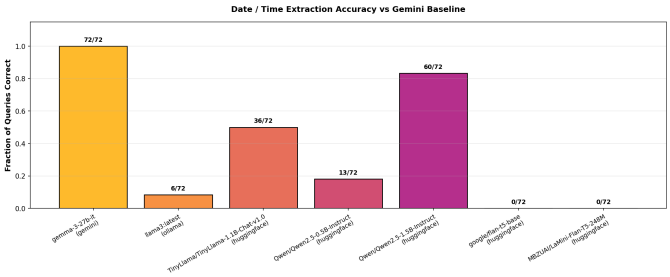

Fig. 1. Keyword Jaccard similarity relative to the Gemini baseline per model. `Qwen2.5-0.5B` retains 71% of Gemini keyword overlap; all other local models converge to zero.

interactive use. This confirms the scaling law prediction: the 1.5B model crosses a reasoning threshold the 0.5B cannot, but does so at a cost that rules it out without GPU acceleration or quantization.

TABLE IV
SCALE ABLATION: 8-QUERY EXPLORATORY EVALUATION

Model	Date Matches	Time Matches	Latency (s)
`Qwen2.5-0.5B-Instruct`	1/7	0/2	≈3.2
`Qwen2.5-1.5B-Instruct`	3/7	2/2	≈129
`gemma-3-27b-it`	**7/7**	**2/2**	≈7.8

H. Abandoned Candidates

Two model families were evaluated and abandoned before the rigorous benchmark:

a) TIMEX3 Sequence Labeler (`temporal_tagger_roberta2roberta`): This 250M encoder-decoder was fine-tuned for TIMEX3 annotation of news documents. Applied to query-side temporal resolution, it produced bare `<TIMEX3>` tags anchored to document

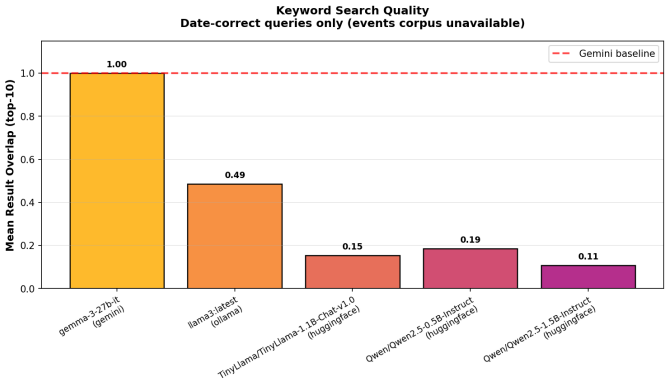

Fig. 2. Temporal accuracy: date exact-match and time exact-match rates per model. Only Gemini achieves non-zero accuracy on both fields.

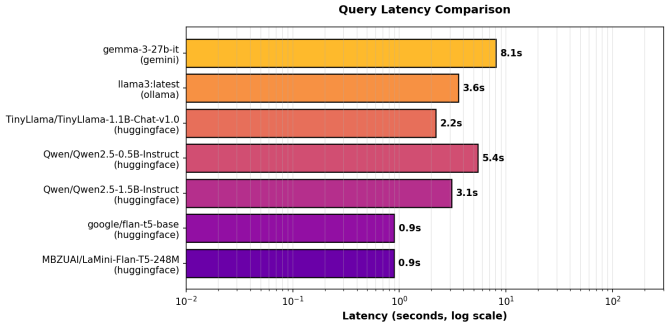

Fig. 3. Latency CDF across all 72 benchmark queries. T5 variants are fastest; decoder-only local models cluster between 6–24 s.

context rather than the user's present date, inflating every query to a full-year range or hallucinating year values (e.g., 1998-W44). Result: 0/8 date matches. The domain gap between document-centric tagging and query-side resolution against a live reference date is fundamental.

b) Zero-Shot Seq2Seq (`flan-t5-base`): Zero-shot prompting for JSON output produced alphabetic enumeration or input token repetition. The model has no chat template; structured generation at 250M parameters without fine-tuning is unreliable. Although it achieves 100% parse success in the rigorous benchmark (seq2seq output is syntactically valid), it scores zero on all substantive metrics.

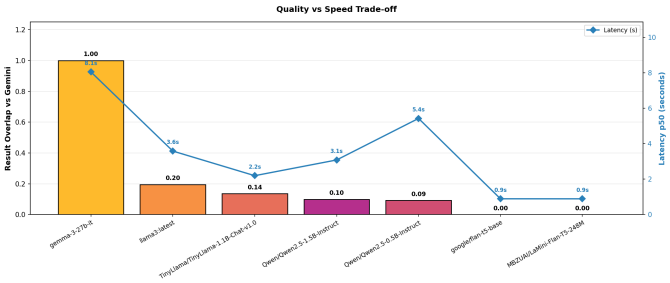

Fig. 4. Quality (MRR) vs. speed (mean latency). Gemini occupies the quality frontier; no local model approaches it despite comparable or lower latency.

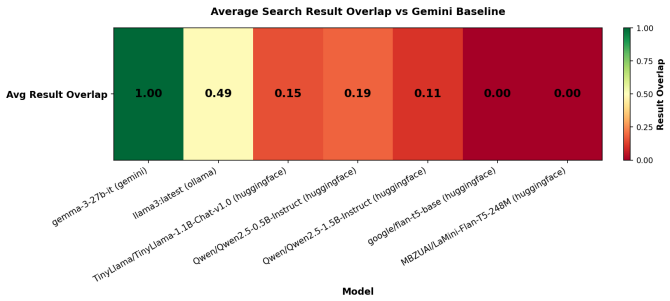

Fig. 5. Per-query score heatmap (rows: queries; columns: models). The Gemini column is dense; all other columns are sparse or empty.

I. Error Analysis

Table V summarizes the primary failure mode for each model across all 72 benchmark queries. We identify four distinct failure classes that partition the local model space.

TABLE V
ERROR TAXONOMY ACROSS ALL MODELS (N=72 PER MODEL)

Model	Failure Mode	Count	Rate	Root Cause
`gemma-3-27b-it`	None	0	0%	—
`Qwen2.5-0.5B-Instruct`	Truncated JSON	55	76.4%	Token limit reached mid-string
`Qwen2.5-1.5B-Instruct`	Truncated JSON	72	100%	Token limit reached mid-string
`TinyLlama-1.1B`	Empty output	72	100%	No tokens generated after prompt
`flan-t5-base`	Semantic failure	0	0%	Parses but all fields empty
`LaMini-Flan-T5-248M`	Non-JSON prefix	72	100%	Non-JSON first token

a) Truncated JSON (`Qwen2.5-0.5B`, `Qwen2.5-1.5B`): Both Qwen models begin generating syntactically valid JSON but exhaust the 512-token budget before closing all string fields, producing unterminated-string errors on parse. At 0.5B scale, 55 of 72 queries (76.4%) trigger this; at 1.5B, all 72 do—a counterintuitive regression explained by the larger model generating longer chain-of-thought reasoning traces before the JSON body, consuming the token budget earlier. Increasing `max_new_tokens` beyond 512 is a likely mitigation.

b) Empty Output (`TinyLlama-1.1B`): TinyLlama produces no tokens after the prompt on every query, yielding a `line 1 column 1` parse error. The model's chat template appears incompatible with the few-shot `user/assistant` format: it either echoes the prompt without generating a response or produces only whitespace. This is a prompt-format mismatch rather than a capacity failure.

c) Non-JSON Prefix (`LaMini-Flan-T5-248M`): LaMini returns an `Extra data: line 1 column 2` error on all queries, indicating the output starts with a single non-JSON character before additional content. The model does not follow the JSON output format at all under zero-shot prompting, consistent with its T5 seq2seq heritage and the lack of a chat template.

d) Semantic Failure (`flan-t5-base`): Unlike the other local models, `flan-t5-base` always produces syntactically parseable output (100% parse success), but every predicted field is empty. The model returns the correct JSON skeleton with no values populated—it has learned the output schema structure from its FLAN instruction-tuning but cannot populate field values from the query at 250M parameters.

e) Within-Parse-Success Errors (`Qwen2.5-0.5B`): Of the 17 queries where `Qwen2.5-0.5B` successfully parses, 14 produce correct date bounds (82%) but only 8 produce correct time bounds (47%). Remaining temporal errors fall into two patterns:

- **Week boundary confusion**: "this week" anchors to the current weekday rather than the Monday–Sunday span.
- **Day-of-week arithmetic**: "next Thursday" returns today's date when today is a weekday, indicating failure to advance to the next occurrence.

These are semantic reasoning failures that occur even when the model generates valid JSON, and are not recoverable by the `dateparser` fallback since the model's output is well-formed but wrong.

V. CONCLUSION

We present Curia-Lite, an efficiency study on model-size minimization for event query grounding. Our results confirm asymmetric degradation: under a consistent in-context learning protocol and fallback policy, keyword extraction retains partial function at 500M parameters while temporal structured output is essentially zero below the 27B scale on consumer CPU. A sub-1B structured-output ceiling is real and does not appear bridgeable by prompt engineering alone without quantization or GPU inference.

Key findings:

- **Keyword extraction is more size-resilient**: At 500M parameters, keyword F1 retains 71% of the 27B baseline. Temporal extraction retains 0%.
- **A structured-output ceiling exists below 1B parameters on CPU**: No model smaller than 27B produces consistent parseable JSON with correct temporal arithmetic across the 72-query benchmark.
- **The 1.5B→0.5B step is discontinuous for temporal reasoning**: The 1.5B model partially solves problems the 0.5B cannot, but at 14× the latency, making it impractical without hardware acceleration.
- **Hybrid retrieval is an efficient supplement**: MiniLM at 22M parameters and <50 ms augments keyword output for all model sizes with negligible overhead.

- **Fallbacks cannot close the gap**: The same `dateparser` fallback chain applies at every scale. It recovers explicit date strings but does not recover relative or compound temporal reasoning.

A. Future Work

- **Quantized local models**: GGUF int4/int8 variants of Qwen2.5-1.5B and Qwen2.5-3B to improve the local accuracy-latency frontier without full-precision CPU inference.
- **Confidence-gated fallback**: Use token log-probabilities to decide when to trust LLM output versus fall back to rules, reducing unnecessary fallback invocations.
- **Recurring event queries**: Extend temporal extraction to handle recurring expressions ("every Monday", "weekly seminars").
- **Expanded benchmark**: Add robustness (typo) and cross-domain variants to cover the full three-dataset suite (*retrieval*, *temporal*, *robustness*).

References

[1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[2] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[3] M. Verhagen, R. Gaizauskas, F. Schilder, M. Hepple, G. Katz, and J. Pustejovsky, "SemEval-2007 task 15: TempEval temporal relation identification," in *Proceedings of the Fourth International Workshop on Semantic Evaluations (SemEval-2007)*, 2007, pp. 75–80.

[4] J. Strötgen and M. Gertz, "HeidelTime: High quality rule-based extraction and normalization of temporal expressions," in *Proceedings of the 5th International Workshop on Semantic Evaluation*, 2010, pp. 321–324.

[5] J. Pustejovsky, P. Hanks, R. Saurí, A. See, R. Gaizauskas, A. Setzer, D. Radev, B. Sundheim, D. Day, L. Ferro *et al.*, "The TimeBank corpus," in *Corpus Linguistics*, vol. 2003, 2003, pp. 647–656.

[6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

[7] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "LlaMa 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[8] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[10] dateparser contributors, "dateparser: Python parser for human readable dates," https://github.com/scrapinghub/dateparser, 2023.